

CDSTR Reference Manual

Clojure Front End, Shell, and Tooling

Sambuddha Majumder and Jayanta Majumder

May 24, 2026

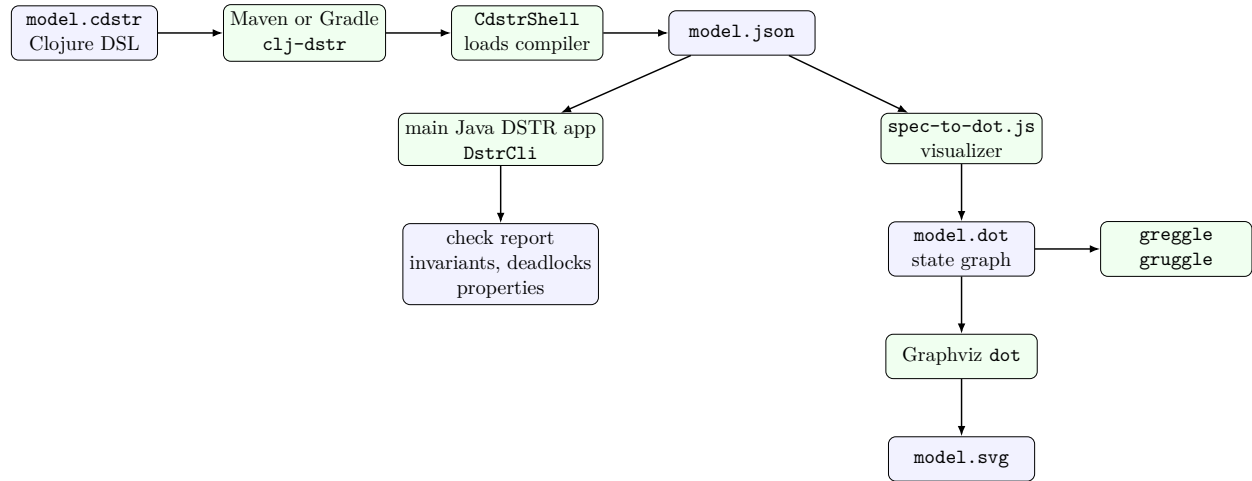
Contents

1 Purpose and Scope

CDSTR is the Clojure-shaped front end for **dstr**. It reads **.cdstr** files, macroexpands their DSL forms with Clojure, and emits the same normalized JSON format used by the core checker and graph tools. The front end lives in the standalone **clj-dstr** Java project and can be run with Maven or Gradle. Clojure is obtained from Maven repositories, so no separate Clojure installation is required beyond the Java build tool.

CDSTR is useful when a model benefits from homoiconicity and Clojure macros but should still target the same explicit-state JSON as DSTR and TDSTR.

2 Tooling Layout



The Clojure project is only the front end: it turns **.cdstr** into the shared JSON representation. The main model-checking step is performed by the root Java DSTR application, **org.dstr.cli.DstrCli**. Use that checker for invariant checking, deadlock detection, transition counts, and **eventually** property results. Use **spec-to-dot.js** and Graphviz when you also want a visual state graph. The generated **.dot** graph can also serve as input to **greggle** and **gruggle** for graph interrogation and manipulation.

3 Command Reference

3.1 Repository Wrappers

From the repository root, use `cdstr.bat` or `cdstr.sh` to compile a single `.cdstr` file to JSON.

```
.\cdstr.bat cdstr-testsuite\light-switch.cdstr
.\cdstr.bat cdstr-testsuite\light-switch.cdstr target\light-switch.json
```

```
./cdstr.sh cdstr-testsuite/light-switch.cdstr
./cdstr.sh cdstr-testsuite/light-switch.cdstr target/light-switch.json
```

If the output path is omitted, the wrapper writes a JSON file adjacent to the source file.

3.2 SVG Wrapper

```
.\cdstr-to-svg.bat cdstr-testsuite\light-switch.cdstr
.\cdstr-to-svg.bat cdstr-testsuite\bakery-3proc.cdstr --max-states 500
```

```
./cdstr-to-svg.sh cdstr-testsuite/light-switch.cdstr
./cdstr-to-svg.sh cdstr-testsuite/bakery-3proc.cdstr --max-states 500
```

The SVG wrapper accepts `input.cdstr` or `input.json`. For source input it produces adjacent `.json`, `.dot`, and `.svg` files. For JSON input it reuses the normalized spec directly. This wrapper is for visualization; run the Java model checker when the goal is a checking report. Use the `.svg` output for viewing and the `.dot` output as input to `greggle` or `gruggle`.

3.3 Run the Java Model Checker

After producing JSON with `cdstr`, run the root Java DSTR application:

```
mvn exec:java -Dexec.args="cdstr-testsuite/light-switch.json"
gradle run --args="cdstr-testsuite/light-switch.json"
```

The checker enumerates the finite state space, starts from all states satisfying `init`, follows the `next` relation, checks invariants on reachable states, records deadlock paths, and reports existential reachability results for `eventually` properties. Its output is the authoritative check result for the normalized JSON produced by CDSTR.

3.4 Running the Standalone Project

The Clojure front end can also be run from inside `clj-dstr`.

```
cd clj-dstr
mvn compile exec:java -Dexec.args="samples/light-switch.cdstr target/light-switch.json"
```

```
cd clj-dstr
gradle run --args="samples/light-switch.cdstr target/light-switch.json"
```

Directory inputs are accepted:

```
mvn -q -f clj-dstr/pom.xml compile exec:java \
  "-Dexec.args=cdstr-testsuite target/cdstr-generated"
```

4 Reader Model

CDSTR source is read by the Clojure reader. That has several practical consequences:

- Parenthesized forms are ordinary Clojure lists.
- Clojure `defn` and `defmacro` can appear before the `system` form.
- Clojure comments beginning with `;` are available.
- Symbols such as `light+` and `$turn-on` are legal Clojure symbols and are interpreted by the CDSTR compiler.
- The reader macro `@x` means Clojure dereference. Prefer `$action` or `(action-ref action)` for action references.

5 System Form

```
(system light-switch
  (vars light)
  (domain light off on)
  (init (= light off))
  (action turn-on
    (= light off)
    (= light+ on))
  (action turn-off
    (= light on)
    (= light+ off))
  (next (or $turn-on $turn-off))
  (invariant type-ok
    (in light (set off on)))
  (property eventually-on
    (eventually (= light on))))
```

The top-level system clauses are:

vars Declare state variables.

vars* Declare a Cartesian product of variable names.

domain Declare one finite domain.

domain* Declare domains for variables matching a glob pattern.

init Define initial-state predicate.

action Define a named transition predicate.

next Define the next-state relation.

invariant Define a predicate checked on reachable states.

property Define a property summary, commonly with **eventually**.

Multiple body forms in a clause are compiled as an implicit conjunction.

6 Atoms and References

Current variable A declared symbol such as `pc`.

Next variable A declared symbol with `+`, such as `pc+`.

Action reference `$p1-start`, `(action-ref p1-start)`, or Clojure's dereference form if it reaches the compiler as `(deref p1-start)`. New CDSTR models should use `$action`.

Domain literal An undeclared symbol in a domain body becomes a string literal.

Expression literal An undeclared symbol in expressions also becomes a string literal unless it is a local quantified variable.

Booleans Clojure `true`, `false`, and `nil` are accepted; `t` is also accepted as true for lispy parity.

Numbers and strings Clojure numbers and strings become JSON numeric and string literals.

7 Expression Reference

7.1 Logic

```
(and (= pc idle) (= lock free))
(or $enter $exit)
(not (= pc cs))
(implies (= pc cs) (= lock held))
```

7.2 Comparison and Arithmetic

```
(= hr 12)
(!= mem 0)
(< ticket other-ticket)
(= hr+ (if-wrap (+ hr 1))) ; user macro/function may produce DSL
(= next-count (+ count 1))
```

The DSL operators include `=`, `!=`, `<`, `<=`, `>`, `>=`, `+`, `-`, `*`, and `/`.

7.3 Sets

```
(domain mode idle running stopped)
(in mode (set idle running stopped))
(= allowed (set p1 p2 p3))
```

7.4 Quantifiers

```
(forall (p in (set p1 p2))
  (in p (set p1 p2)))

(exists (candidate in (set p1 p2))
  (= candidate owner))
```

7.5 Reachability-Style Properties

```
(property can-measure-four-gallons  
  (eventually (= big 4)))
```

`eventually` is interpreted by the prototype as “some reachable state satisfies this predicate.”

8 Transition Sugar

8.1 Assignment

All three forms assert equality between the next state of a variable and a value:

```
(= light+ on)  
(assign on to light)  
(set light as on)
```

8.2 Equality Alias

```
(equals master_lifecycle ENRICHING)
```

is equivalent to:

```
(= master_lifecycle ENRICHING)
```

8.3 Guarded Cases

```
(if  
  (equals master_lifecycle ENRICHING)  
  (equals venue1_lifecycle ENRICHING)  
  then  
    (assign READY to master_lifecycle)  
    (assign READY to venue1_lifecycle)  
    (assign NOT_TRADING to master_trading_status)  
    (assign NOT_TRADING to venue1_trading_status))
```

The CDSTR compiler accepts multiple condition forms before `then` and multiple consequence forms after `then`. The result is a conjunction of all condition and consequence forms. There is no `else` branch.

8.4 Frame Conditions

```
(unchanged* *_market_status *_event_type *_mission_version)
```

This emits frame equalities for every declared variable matched by the glob patterns.

8.5 Scenario Alias

```
(alternate-scenarios  
  (if (= state OPEN) then (assign CANCELLED to state))  
  (if (= state BOOKED) then (assign state to state)))
```

`alternate-scenarios` is a synonym for `or`.

9 Clojure Macros

The `system` macro captures the unevaluated DSL body, then the compiler macroexpands recognized non-reserved Clojure macro calls before normalizing the model. A macro can return one DSL form or a sequence of sibling forms.

9.1 Built-In Helpers

```
(same pc)           ; expands to (= pc+ pc)
(unchanged p2pc mem) ; expands to sibling frame clauses
(next-var-symbol 'pc) ; returns symbol pc+
```

9.2 Simple Clojure Macro

```
(defmacro preserve [& vars]
  (mapv (fn [v] (list '(next-var-symbol v) v)) vars))

(system example
  (vars pc mem)
  (domain pc idle done)
  (domain mem 0 1)
  (init (= pc idle) (= mem 0))
  (action finish
    (= pc idle)
    (= pc+ done)
    (preserve mem))
  (next $finish))
```

Here `preserve` returns sibling frame clauses. The compiler splices those clauses into the containing action body.

9.3 Including Helper Files

```
(include "helpers.clj")

(system uses-helper
  ...)
```

Relative include paths are resolved against the directory of the current `.cdstr` file.

10 Product Variables and Large Models

```
(system trade-cancellation-parent-children-small
  (vars* _ (p c1 c2) * (market_status settlement_state settlement_version
    settlement_revision event_type))

  (domain* *_market_status MATCHED)
  (domain* *_settlement_state OPEN RELEASED BOOKED CANCELLED)
  (domain* *_event_type INSTRUCTION_ACKED)
  (domain* *_mission_version 1))
```

```

(domain* *_mission_revision 1)

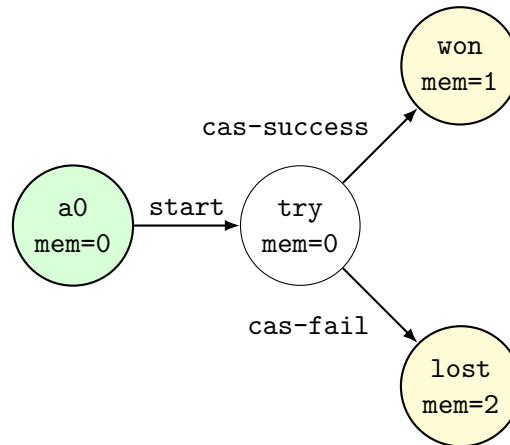
(init
  (equals p_settlement_state OPEN)
  (equals c1_settlement_state OPEN)
  (equals c2_settlement_state OPEN))

(action trade-cancellation
  (unchanged* *_market_status *_event_type *_mission_version *_mission_revision)
  (alternate-scenarios
    (if
      (equals p_settlement_state OPEN)
      (equals c1_settlement_state OPEN)
      (equals c2_settlement_state OPEN)
      then
        (assign CANCELLED to p_settlement_state)
        (assign CANCELLED to c1_settlement_state)
        (assign CANCELLED to c2_settlement_state))))
  (next $trade-cancellation))

```

Use `-max-states` when visualizing large explicit-domain variants.

11 State Graph Interpretation



The Graphviz output labels reachable counts, visible transitions, deadlocks, invariant failures, actions, and property truth values. Compact graphs move the action and property summaries into a legend node.

12 CDSTR Test Suite

The `cdstr-testsuite` directory mirrors the DSTR and TDSTR examples. It includes:

- `light-switch.cdstr`
- `tla-hour-clock.cdstr`

- `tla-die-hard.cdstr`
- `tla-peterson-2proc.cdstr`
- `bakery-3proc.cdstr`
- `cas-2proc-race.cdstr`
- `instrument-lifecycle.cdstr`
- `instrument-lifecycle-small.cdstr`
- `trade-cancellation-parent-children.cdstr`
- `trade-cancellation-parent-children-small.cdstr`

Compile the suite:

```
mvn -q -f clj-dstr/pom.xml compile exec:java \
"-Dexec.args=cdstr-testsuite target/cdstr-generated"
```

Then pass any generated JSON file to the root Java checker:

```
mvn exec:java -Dexec.args="target/cdstr-generated/light-switch.json"
```

13 Practical Notes

- Prefer `$action` for action references in CDSTR. It is portable across the lispy front ends and avoids Clojure reader-deref ambiguity.
- Keep user macro names distinct from reserved DSL heads such as `action`, `if`, `set`, and `forall`.
- A macro that returns sibling clauses should return a sequence of forms, not a quoted wrapper form.
- Next-state variables are only meaningful in transition contexts: `action` and `next`.
- Clojure code runs at compile time. Keep helper macros deterministic so generated JSON remains reproducible.
- Treat CDSTR as the source-language layer and the root Java DSTR application as the checking layer. The SVG tooling is for graph inspection.